

bank.c

```
#include <stdio.h>
#include "types.h"
#include "functions.h"

/* Rule-LK: each colour group has a base price that is used ONLY for
loan calculations (individual purchase prices are used for
buying). It matches the first property in the group on the board,
so this reads it straight from the board instead of storing a
second copy. Railways and utilities use their own mortgage value. */
int groupBasePrice(GameState game[], PropertyGroup group)
{
    int i;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type == PROPERTY &&
           game[0].board[i].property.group == group)
        {
            return game[0].board[i].property.purchasePrice;
        }
    }

    return 0;
}

int totalEligibleCollateral(GameState game[], int playerIndex)
{
    int i;
    int total;
    int group;
    int mult;

    total = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY &&
           game[0].board[i].type != RAILWAY &&
           game[0].board[i].type != UTILITY)
        {
            continue;
        }

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.mortgaged)
            continue;

        if(game[0].board[i].property.loanLocked)
            continue;

        /* Rule-LK 34: a booming group's collateral is worth 15% more,
        a declining group's 10% less. Colour property collateral
        is based on the group base price (not the individual
        purchase price - that is only used for buying). */
        group = groupOf(game, i);

        mult = modifierMultiplier(game, MOD_MARKET_MORTGAGE, group, -1);
```

```

        if(game[0].board[i].type == PROPERTY)
            total += (groupBasePrice(game, group) * mult) / 100;
        else
            total += (game[0].board[i].property.mortgageValue * mult) / 100;
    }

    return total;
}

/* Rule-LK 2: max loan is 75% of eligible collateral. */
int calculateMaxLoan(GameState game[], int playerIndex)
{
    return (totalEligibleCollateral(game, playerIndex) * LOAN_COLLATERAL_PERCENT) / 100;
}

void obtainLoan(GameState game[], int playerIndex)
{
    int i;
    int maxLoan;
    char loanBuf[32];

    if(game[0].players[playerIndex].loan.active)
        return; /* only one active loan allowed at a time */

    maxLoan = calculateMaxLoan(game, playerIndex);

    if(maxLoan <= 0)
        return;

    game[0].players[playerIndex].loan.active = 1;
    game[0].players[playerIndex].loan.amount = maxLoan;
    game[0].players[playerIndex].loan.interestRate = game[0].economy.loanInterestRate;
    game[0].players[playerIndex].loan.remainingRounds = LOAN_DURATION_ROUNDS;

    receiveMoney(game, playerIndex, maxLoan);

    printf("\n%s obtained a secured loan.\n", game[0].players[playerIndex].name);

    formatLKR(maxLoan, loanBuf);
    printf("Loan Amount : LKR %s.\n", loanBuf);
    printf("Interest Rate : %d%%\n", game[0].economy.loanInterestRate);
    printf("Duration : %d Rounds\n", LOAN_DURATION_ROUNDS);

    printf("Collateral :\n");

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY &&
            game[0].board[i].type != RAILWAY &&
            game[0].board[i].type != UTILITY)
        {
            continue;
        }

        if(game[0].board[i].property.owner == playerIndex &&
            !game[0].board[i].property.mortgaged &&
            !game[0].board[i].property.loanLocked)
        {
            game[0].board[i].property.loanLocked = 1;
            printf("%s\n", game[0].board[i].name);
        }
    }
}

```

```

void repayLoan(GameState game[], int playerIndex, int amount)
{
    int i;

    if(!game[0].players[playerIndex].loan.active)
        return;

    if(amount > game[0].players[playerIndex].loan.amount)
        amount = game[0].players[playerIndex].loan.amount;

    if(game[0].players[playerIndex].cash < amount)
        amount = game[0].players[playerIndex].cash;

    payMoney(game, playerIndex, amount);
    game[0].players[playerIndex].loan.amount -= amount;

    printf("\n%s repaid LKR %d towards their loan.\n",
        game[0].players[playerIndex].name, amount);

    printf("Outstanding Balance : LKR %d\n",
        game[0].players[playerIndex].loan.amount);

    if(game[0].players[playerIndex].loan.amount <= 0)
    {
        resetLoan(game, playerIndex);

        for(i = 0; i < BOARD_SIZE; i++)
        {
            if(game[0].board[i].property.owner == playerIndex)
                game[0].board[i].property.loanLocked = 0;
        }

        printf("%s has fully repaid the loan. Collateral released.\n",
            game[0].players[playerIndex].name);
    }
}

/* Rule-LK 5: landing on the Bank offers repaying part, repaying in
full, or (if no loan yet) obtaining one. With an existing loan a
player can also extend the term, refinance at the current rate, or
borrow more against newly acquired collateral. Only one
transaction happens per visit. */
void handleBankVisit(GameState game[], int playerIndex)
{
    printf("\n%s landed on Bank of Ceylon.\n", game[0].players[playerIndex].name);

    if(game[0].players[playerIndex].loan.active)
    {
        if(wantsToRepayLoan(game, playerIndex))
        {
            int repayAmount;

            /* every strategy's wantsToRepayLoan() only triggers once
            cash can clear the whole loan, so always repay in full */
            repayAmount = game[0].players[playerIndex].loan.amount;

            repayLoan(game, playerIndex, repayAmount);
            return;
        }

        if(wantsIncreaseLoan(game, playerIndex))
        {

```

```

        increaseLoan(game, playerIndex);
        return;
    }

    if(wantsExtendLoan(game, playerIndex))
    {
        extendLoan(game, playerIndex);
        return;
    }

    if(wantsRefinance(game, playerIndex))
    {
        refinanceLoan(game, playerIndex);
        return;
    }

    return;
}

if(wantsLoan(game, playerIndex))
{
    obtainLoan(game, playerIndex);
}
}

/* Rule-LK 5: a player with an active loan may borrow more against
   collateral they've acquired since the loan was taken out. */
void increaseLoan(GameState game[], int playerIndex)
{
    int i;
    int availableCollateral;
    int additional;

    if(!game[0].players[playerIndex].loan.active)
        return;

    availableCollateral = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY &&
           game[0].board[i].type != RAILWAY &&
           game[0].board[i].type != UTILITY)
        {
            continue;
        }

        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(game[0].board[i].property.mortgaged)
            continue;

        if(game[0].board[i].property.loanLocked)
            continue;

        if(game[0].board[i].type == PROPERTY)
        {
            availableCollateral +=
                (groupBasePrice(game, game[0].board[i].property.group) *
                 modifierMultiplier(game, MOD_MARKET_MORTGAGE,
                                     game[0].board[i].property.group, -1)) / 100;
        }
    }
}

```

```

        else
        {
            availableCollateral += game[0].board[i].property.mortgageValue;
        }
    }

    additional = (availableCollateral * LOAN_COLLATERAL_PERCENT) / 100;

    if(additional <= 0)
        return;

    game[0].players[playerIndex].loan.amount += additional;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].type != PROPERTY &&
            game[0].board[i].type != RAILWAY &&
            game[0].board[i].type != UTILITY)
        {
            continue;
        }

        if(game[0].board[i].property.owner == playerIndex &&
            !game[0].board[i].property.mortgaged &&
            !game[0].board[i].property.loanLocked)
        {
            game[0].board[i].property.loanLocked = 1;
        }
    }

    receiveMoney(game, playerIndex, additional);

    printf("\n%s increased their loan by LKR %d.\n",
        game[0].players[playerIndex].name, additional);
    printf("New Outstanding Balance : LKR %d\n",
        game[0].players[playerIndex].loan.amount);
}

/* Rule-LK 5: extend the loan term by 10 rounds. */
void extendLoan(GameState game[], int playerIndex)
{
    if(!game[0].players[playerIndex].loan.active)
        return;

    game[0].players[playerIndex].loan.remainingRounds += LOAN_EXTEND_ROUNDS;

    printf("\n%s extended their loan by %d rounds.\n",
        game[0].players[playerIndex].name, LOAN_EXTEND_ROUNDS);
    printf("New Duration : %d Rounds remaining\n",
        game[0].players[playerIndex].loan.remainingRounds);
}

/* Rule-LK 5: refinance the loan at the current economy interest rate. */
void refinanceLoan(GameState game[], int playerIndex)
{
    if(!game[0].players[playerIndex].loan.active)
        return;

    game[0].players[playerIndex].loan.interestRate = game[0].economy.loanInterestRate;

    printf("\n%s refinanced their loan at the current rate of %d%%.\n",
        game[0].players[playerIndex].name,
        game[0].economy.loanInterestRate);
}

```

```

}

void demolishBuildingsOn(GameState game[], int index)
{
    game[0].board[index].property.houses = 0;
    game[0].board[index].property.hotel = 0;
}

/* Rule-LK 6, 7: if a loan isn't repaid in time, pledged properties
are seized and auctioned. If nothing is left, the player is
declared bankrupt. */
void foreclose(GameState game[], int playerIndex)
{
    int i;
    int ownsAnythingLeft;

    printf("\n%s has defaulted on their loan!\n", game[0].players[playerIndex].name);

    ownsAnythingLeft = 0;

    for(i = 0; i < BOARD_SIZE; i++)
    {
        if(game[0].board[i].property.owner != playerIndex)
            continue;

        if(!game[0].board[i].property.loanLocked)
        {
            ownsAnythingLeft = 1;
            continue;
        }

        printf("%s (pledged collateral) transferred to the Bank.\n",
            game[0].board[i].name);

        demolishBuildingsOn(game, i);

        stripOwnership(game, i);

        adjustOwnedCount(game, playerIndex, i, -1);

        runAuction(game, i, -1);
    }

    resetLoan(game, playerIndex);

    printf("Outstanding debt cleared.\n");

    if(!ownsAnythingLeft && game[0].players[playerIndex].cash <= 0)
    {
        game[0].players[playerIndex].bankrupt = 1;
        printf("%s has no remaining assets and is declared BANKRUPT.\n",
            game[0].players[playerIndex].name);
    }
}

void processLoans(GameState game[])
{
    int i;
    int interest;

    for(i = 0; i < MAX_PLAYERS; i++)
    {
        if(game[0].players[i].bankrupt)

```

```

        continue;

    if(!game[0].players[i].loan.active)
        continue;

    interest = (game[0].players[i].loan.amount * game[0].players[i].loan.interestRate) / 100;
    game[0].players[i].loan.amount += interest;

    game[0].players[i].loan.remainingRounds--;

    printf("\n%s's loan accrued LKR %d interest. New balance : LKR %d\n",
        game[0].players[i].name, interest, game[0].players[i].loan.amount);

    if(game[0].players[i].loan.remainingRounds <= 0)
    {
        foreclose(game, i);
    }
}
}

```